Tutorial 6
Memory management

Jiawei Li (Michael)
Office hours: Tuesday 3:30-5:30pm
Office: PMB426
Email: jiawei.li@nottingham.edu.cn

Buffer

- *A **temporary storage area** is called buffer. All standard input and output devices contain an input and output buffer. In standard C/C++, streams are buffered, for example in the case of standard input, when we press the key on keyboard, it isn't sent to your program, rather it is buffered by operating system till the time is allotted to that program.*

For the example program we run, what are stored in the buffer?

‘a’ ‘\n’ ‘b’ ‘\n’ …

- *‘a’* -> &a;
- *‘\n’* -> &b;
- *‘b’* -> &c;

How to solve it?

Stack and heap

- Local variables and function parameters are allocated on the stack. This is automatically done by the C compiler.
- The stack is fully controlled by the operating system.
- Dynamic memory allocation on the heap needs to be done manually.

Memory allocation

- Allow us to allocate memory dynamically
- When and why should we allocate data on the heap?

```
int main()
{
        int a;
        int* ptr = malloc(sizeof(int));
        …

        free(ptr);
}
```

5 steps to dynamic memory allocation

Step 1. Work out how much memory you need to allocate

– remember the sizeof() operator!

Step 2. Ask for the amount of memory

– use malloc (memory_size)

Step 3. Store the returned pointer e.g.:

```
int* pInt = malloc( sizeof(int) );
```

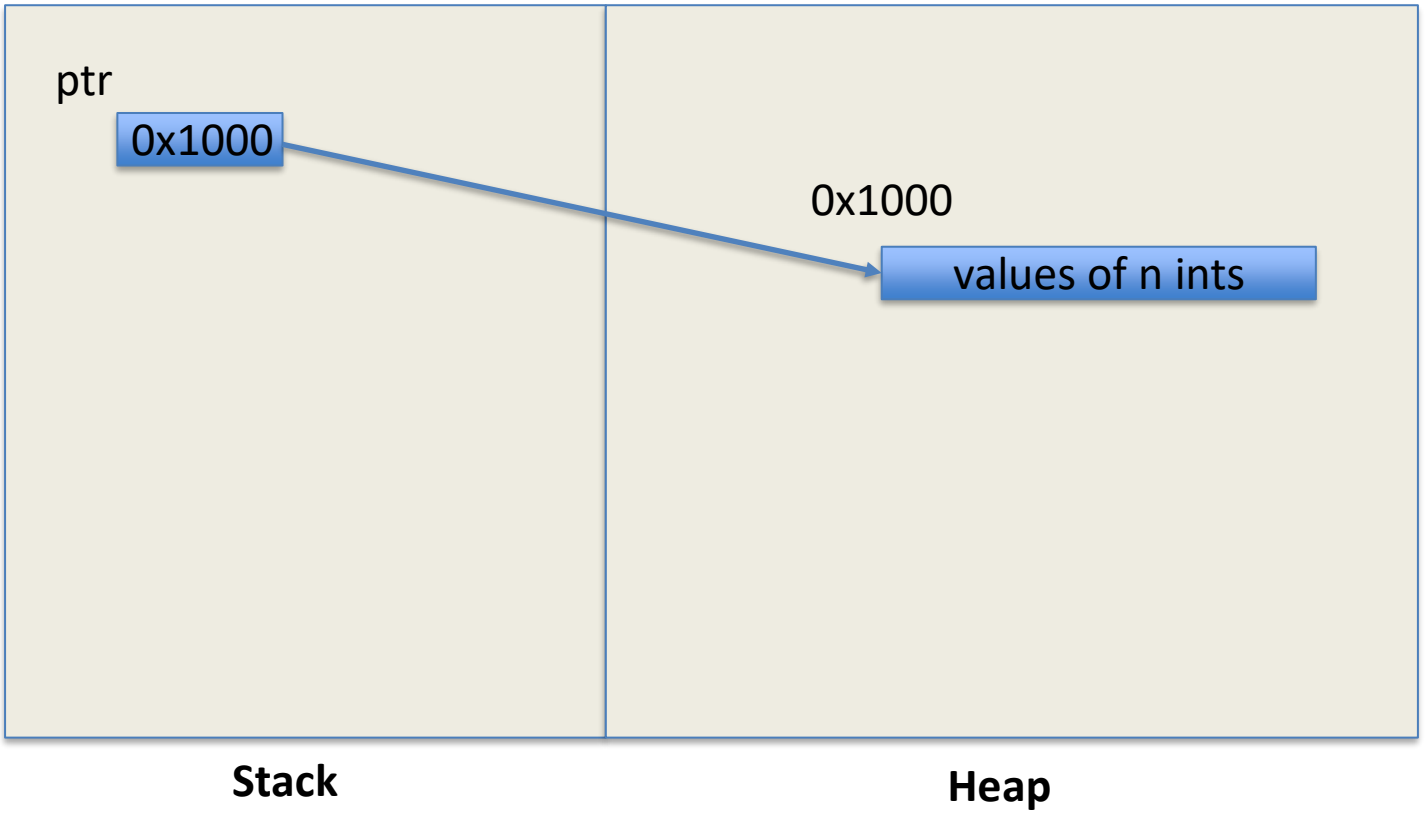
Step 4. Use the memory through the pointer, as if it was the correct type

```
*pInt = 5;   (*pInt)++;   pInt = p;
```

Step 5. when finished, free the memory

```
free( pInt );
```

```
int *ptr;
ptr = malloc(n * sizeof(int));
```



Frequent mistakes

- **Malloc() and free() should be used in pair. Error occurs when we mistakenly free up allocated chunk or we try to free up a part of it.**

```
{       int *ptr;
        int n = 10;

        ptr = malloc(n * sizeof(int));
        …

        free(ptr+5);    // want to free part of ptr

        free(ptr);
        …
        free(ptr);      // free up more than once
}
```

Exercise 1 Figure out what this program does

```
int main()
{
        int i,n;
        float *element;
        scanf(“%d”, &n);
        element = malloc( n*sizeof(float) );
        if(element==NULL) {
                exit(0);
        }
        for(i=0;i<n;++i)
        {
                scanf("%f",element+i);
        }
        for(i=1;i<n;++i)
        {
                if(*element<*(element+i))
                        *element=*(element+i);
        }
        printf(" The wanted element is : %.2f \n\n",*element);
        free(element);
        return 0;
}
```

Exercise 2 Handling unlimited length of input

```
char *prompt()
{
   int cur_array_len = 2;
   int pos = 0;
   char ch;

   char *text = malloc(sizeof(char) * cur_array_len);
   do
   {
      scanf("%c", &ch);
      if(ch == '\n')
                text[pos] = '\0';

      else
      {
                text[pos] = ch;
                pos++;
                if(pos == cur_array_len - 1)
                {
                        // expand array
                        char *t2 = malloc(sizeof(char) * cur_array_len * 2);
                        int i;
                        for(i = 0; i < cur_array_len; i++)
                                t2[i] = text[i];

                        cur_array_len *= 2;
                        free(text);
                        text = t2;
                }
      }
   } while(ch != '\n');
   return text;
}
```

More debugging exercises

Two programs with the names ‘incorrect1.c’ and ‘incorrect3.c’ are available on the Moodle page. Try to find out bugs in two programs. The first program should output the quotient and remainder of the division of two values. The second program is to compare the record of two students (using C struct).